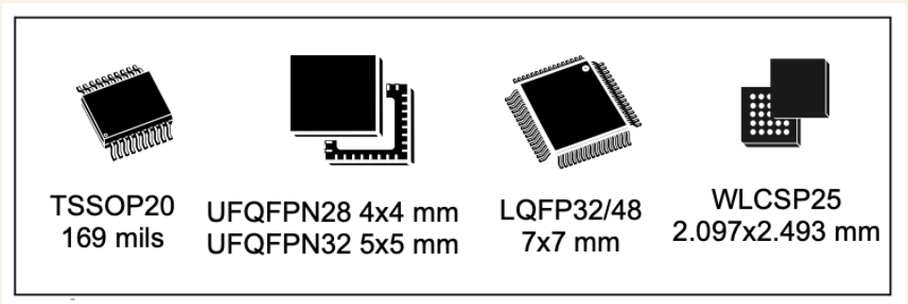
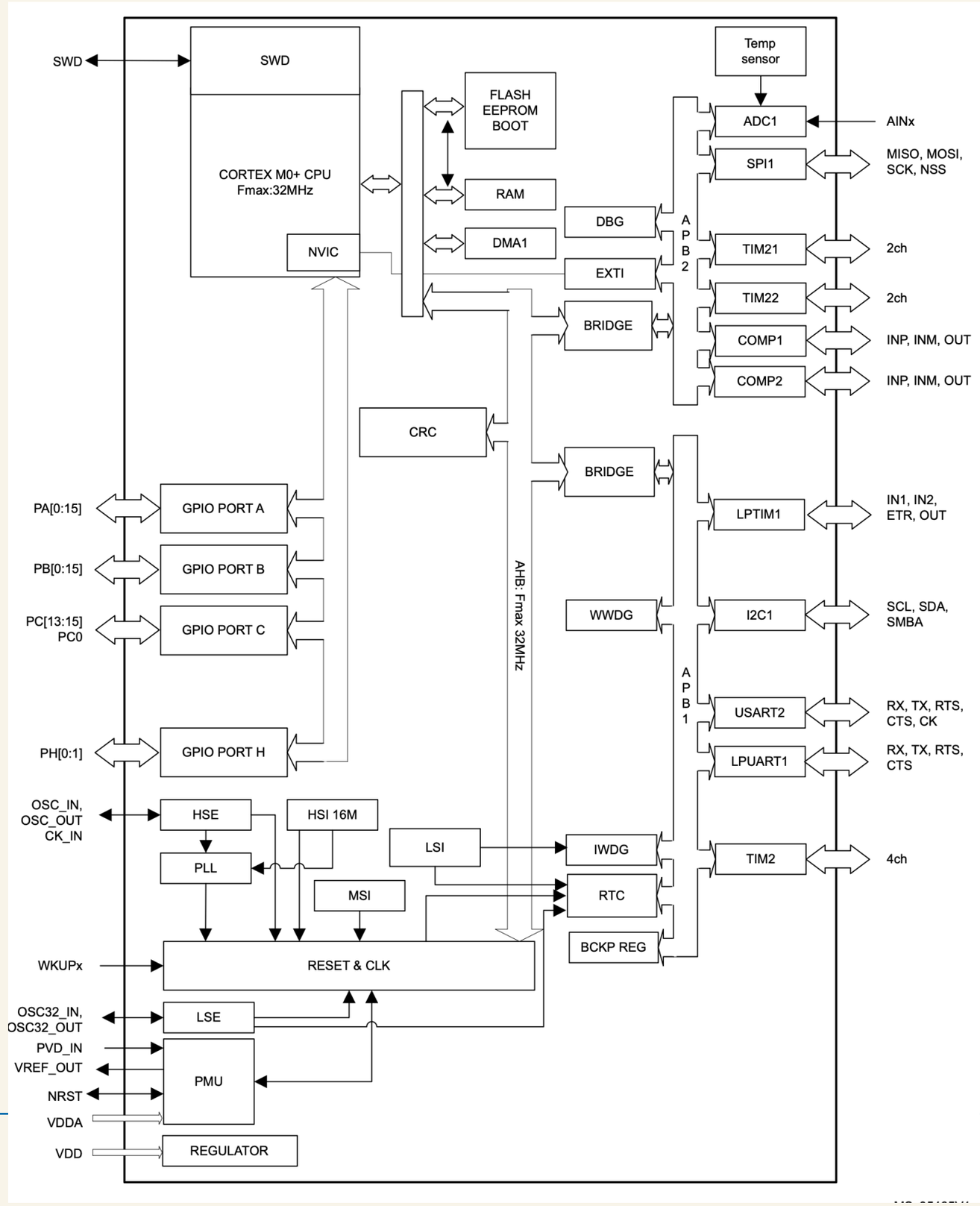
Interfaces

- Microcontroller - STM32
- Memory Map
- CS Logic
- UART
- I2C
- SPI
- MyPI-Nano
- CAN

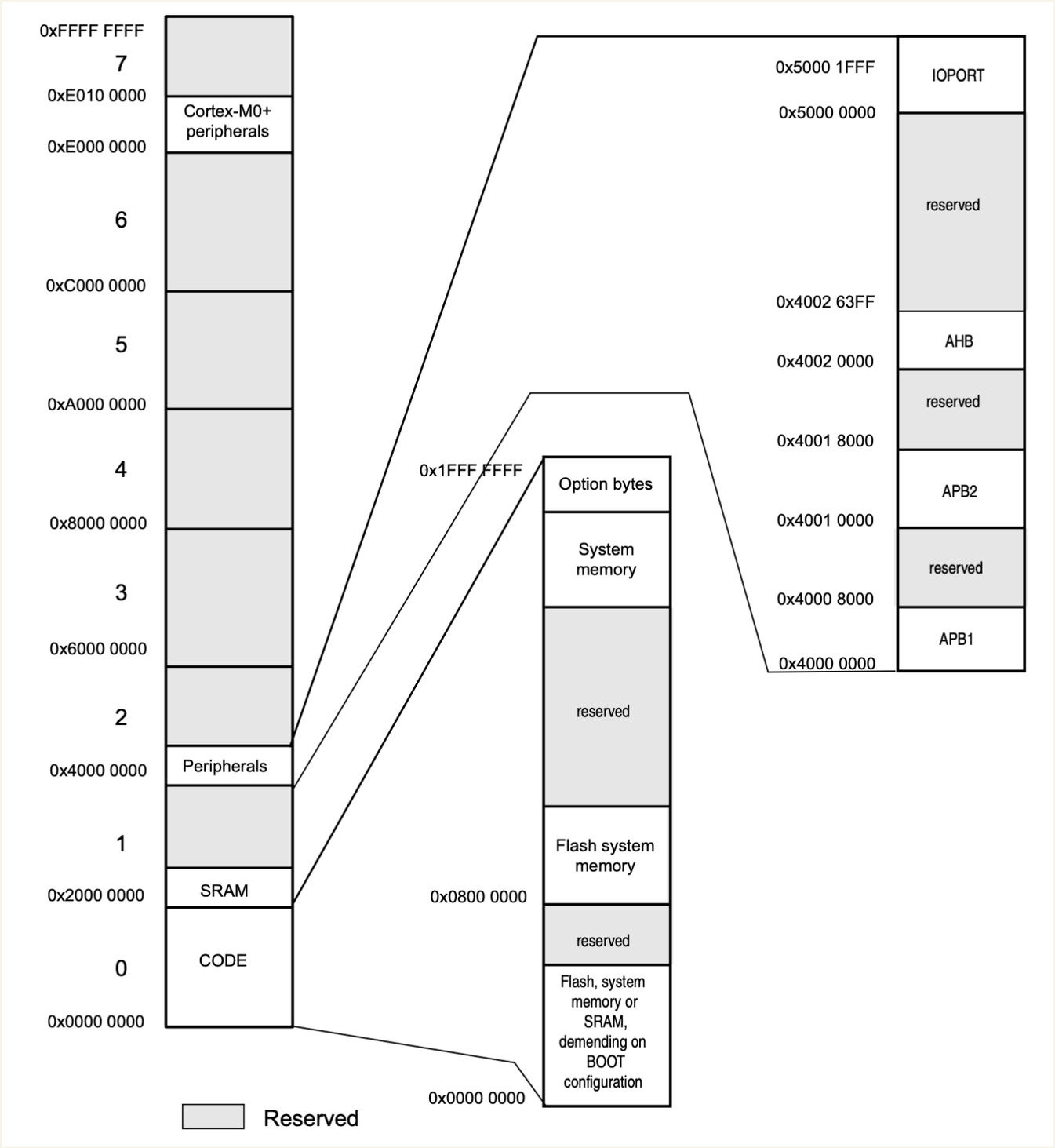
Microcontroller - STM32

- STM32L031x4 STM32L031x6
  - Access line ultra-low-power 32-bit MCU ARM®-based Cortex®-M0+, up to 32KB Flash, 8KB SRAM, 1KB EEPROM, ADC Map

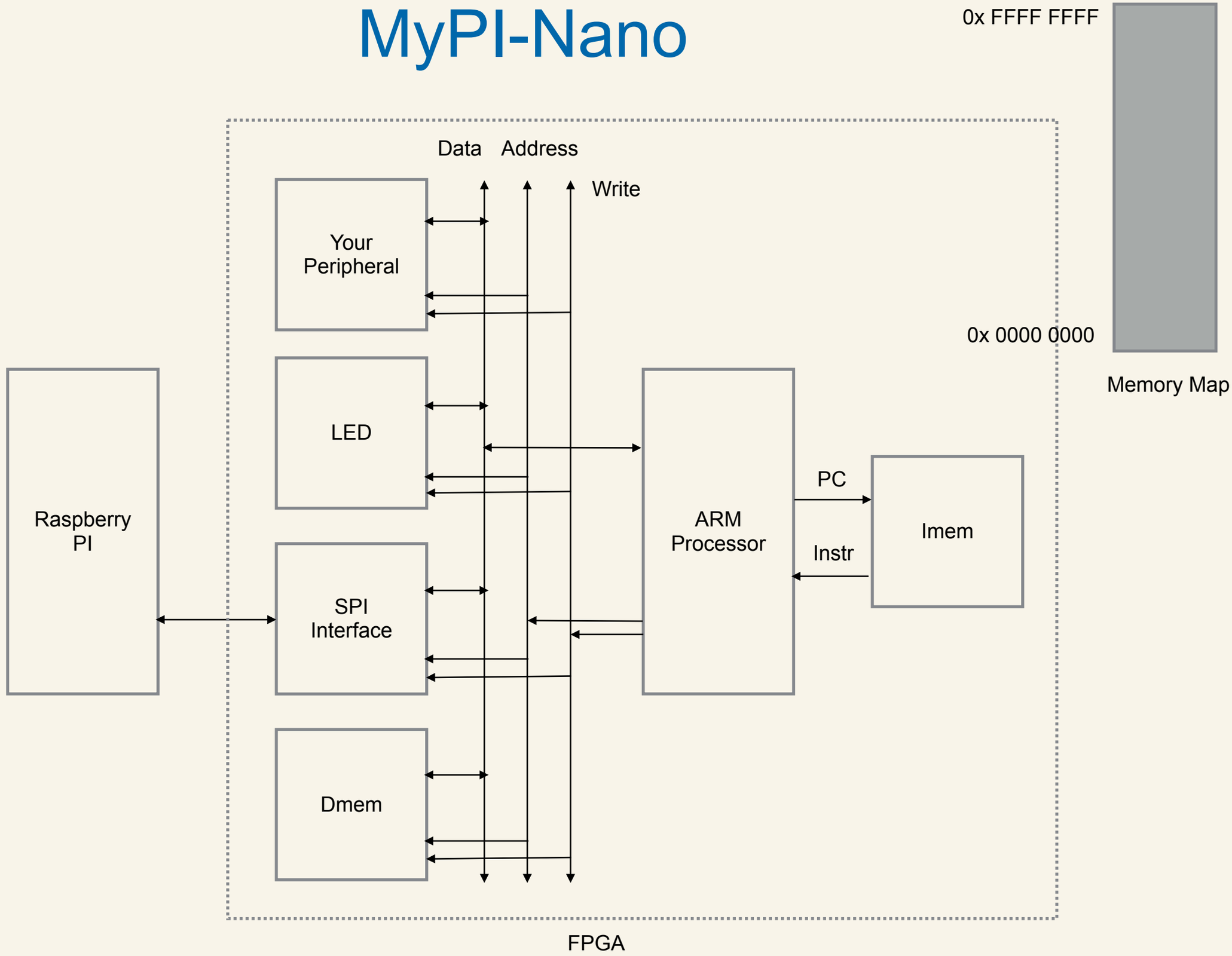




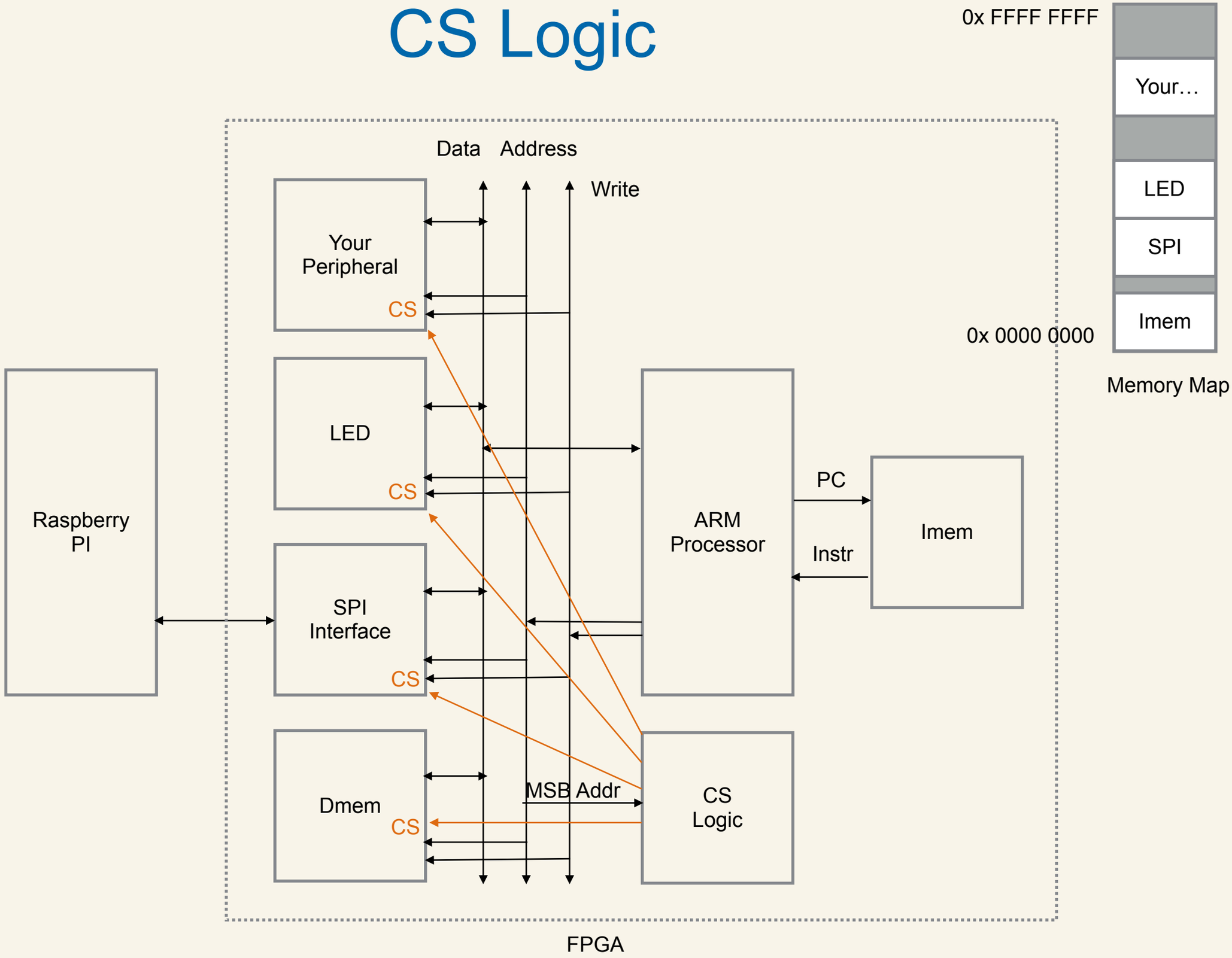
Memory Map


0xFFFF FFFF
7
0xE010 0000
Cortex-M0+ peripherals
0xE000 0000
6
0xC000 0000
5
0xA000 0000
4
0x8000 0000
3
0x6000 0000
2
0x4000 0000
Peripherals
1
0x2000 0000
SRAM
0
CODE
0x0000 0000
0x1FFF FFFF
Option bytes
System memory
reserved
Flash system memory
0x0800 0000
reserved
Flash, system memory or SRAM, demending on BOOT configuration
0x0000 0000
0x5000 1FFF
IOPORT
0x5000 0000
reserved
0x4002 63FF
AHB
0x4002 0000
reserved
0x4001 8000
APB2
0x4001 0000
reserved
0x4000 8000
APB1
0x4000 0000
Reserved

MyPI-Nano

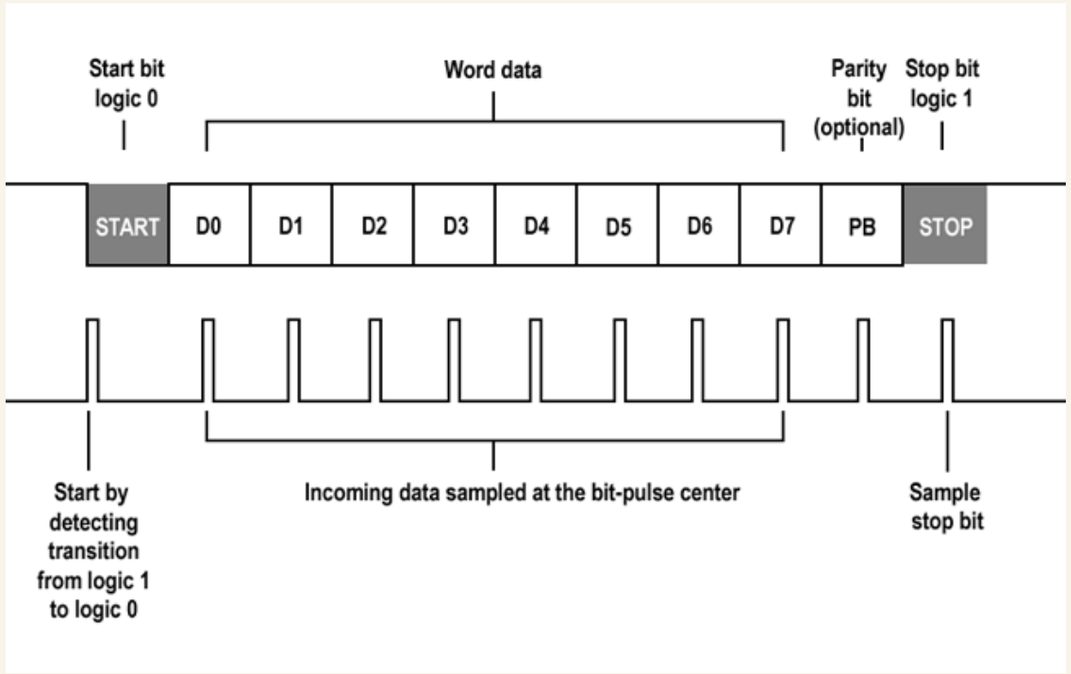

0x FFFF FFFF
0x 0000 0000
Memory Map
Data
Address
Write
Your Peripheral
LED
SPI Interface
Dmem
Raspberry PI
ARM Processor
PC
Instr
Imem
FPGA

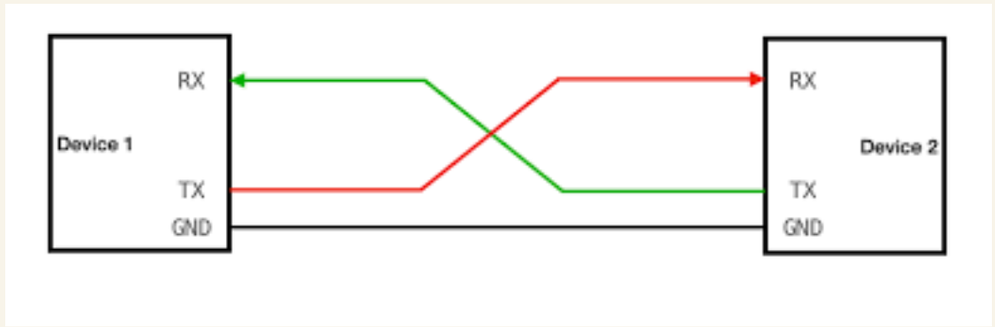
CS Logic

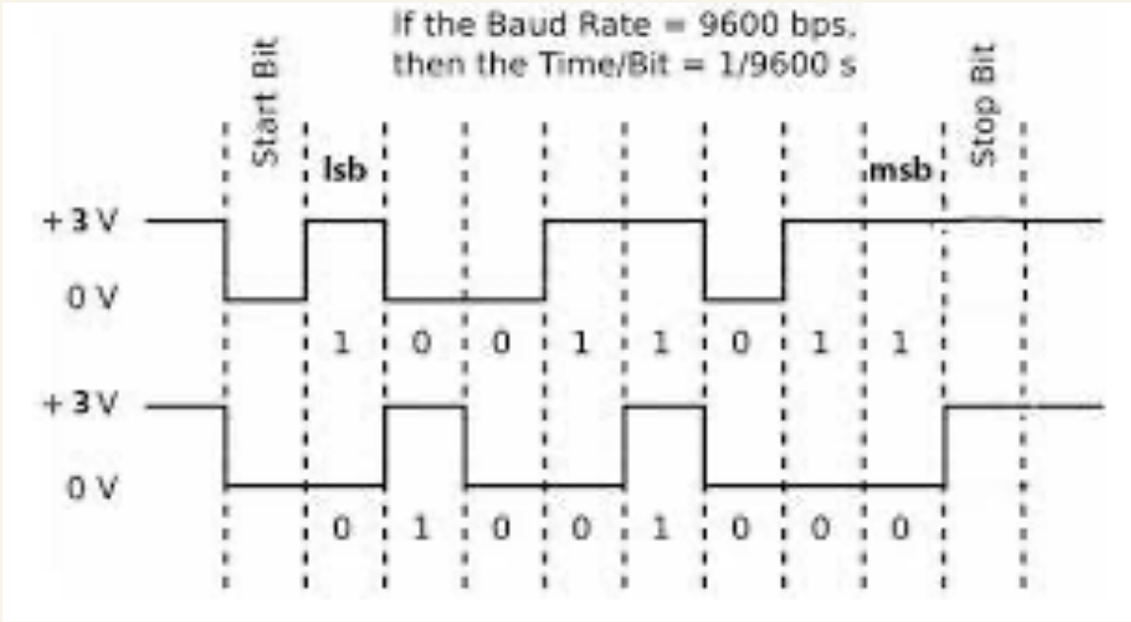

Data
Address
Write
Your Peripheral
CS
LED
CS
SPI Interface
CS
Dmem
CS
MSB Addr
CS Logic
Raspberry PI
ARM Processor
PC
Instr
Imem
FPGA

0x FFFF FFFF
Your…
LED
SPI
Imem
0x 0000 0000
Memory Map

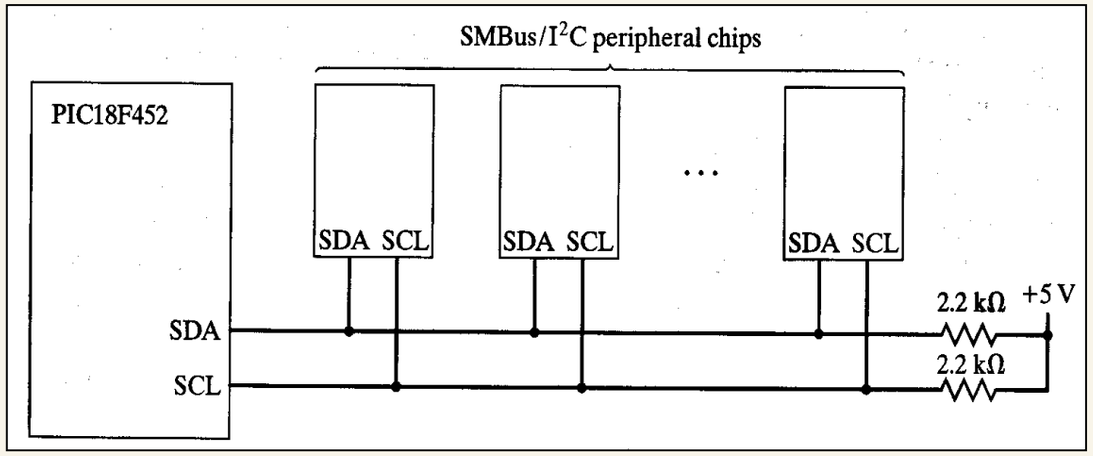
UART

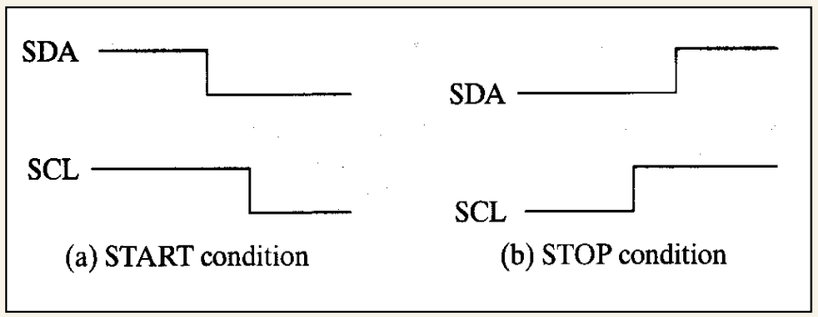

Start bit
logic 0
Word data
Parity
bit
(optional)
Stop bit
logic 1
START
D0
D1
D2
D3
D4
D5
D6
D7
PB
STOP
Start by
detecting
transition
from logic 1
to logic 0
Incoming data sampled at the bit-pulse center
Sample
stop bit

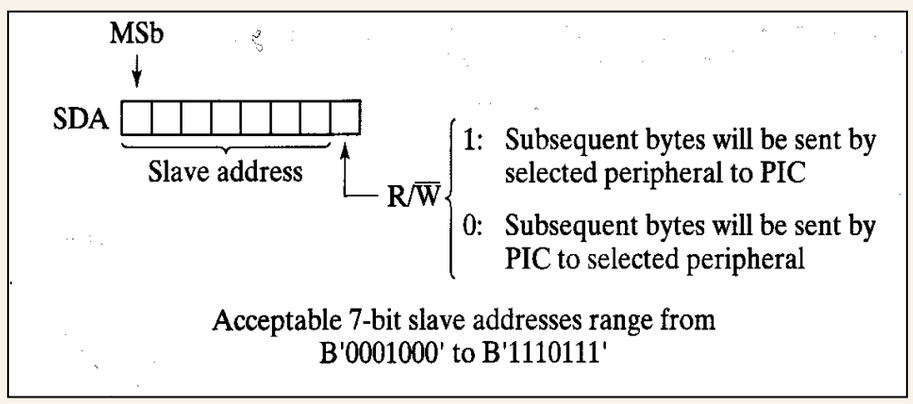

Device 1
RX
TX
GND
RX
TX
GND
Device 2

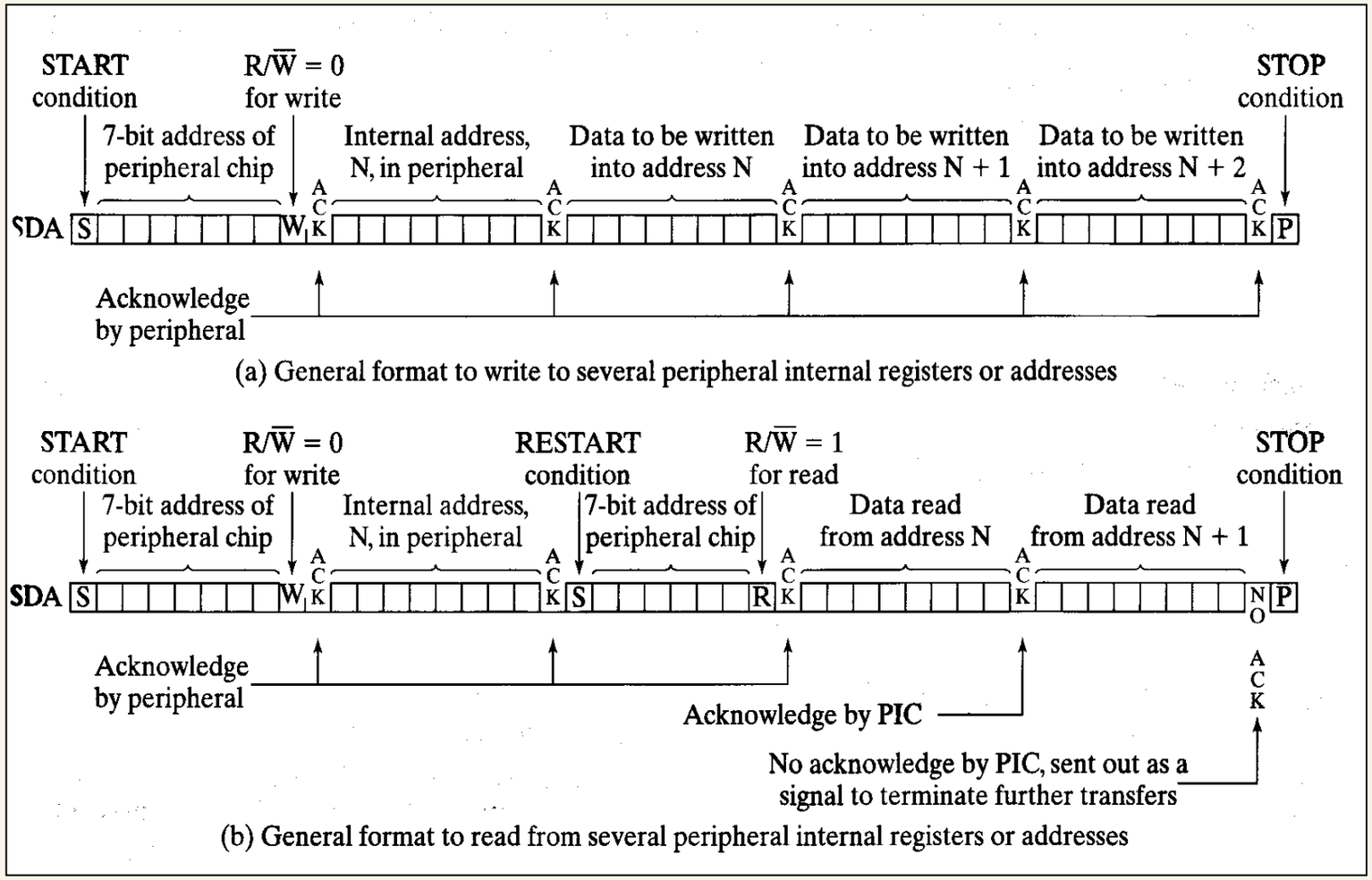

If the Baud Rate = 9600 bps,
then the Time/Bit = 1/9600 s
Start Bit
lsb
msb
Stop Bit
+3 V
0 V
1 0 0 1 1 0 1 1
+3 V
0 V
0 1 0 0 1 0 0 0

I2C

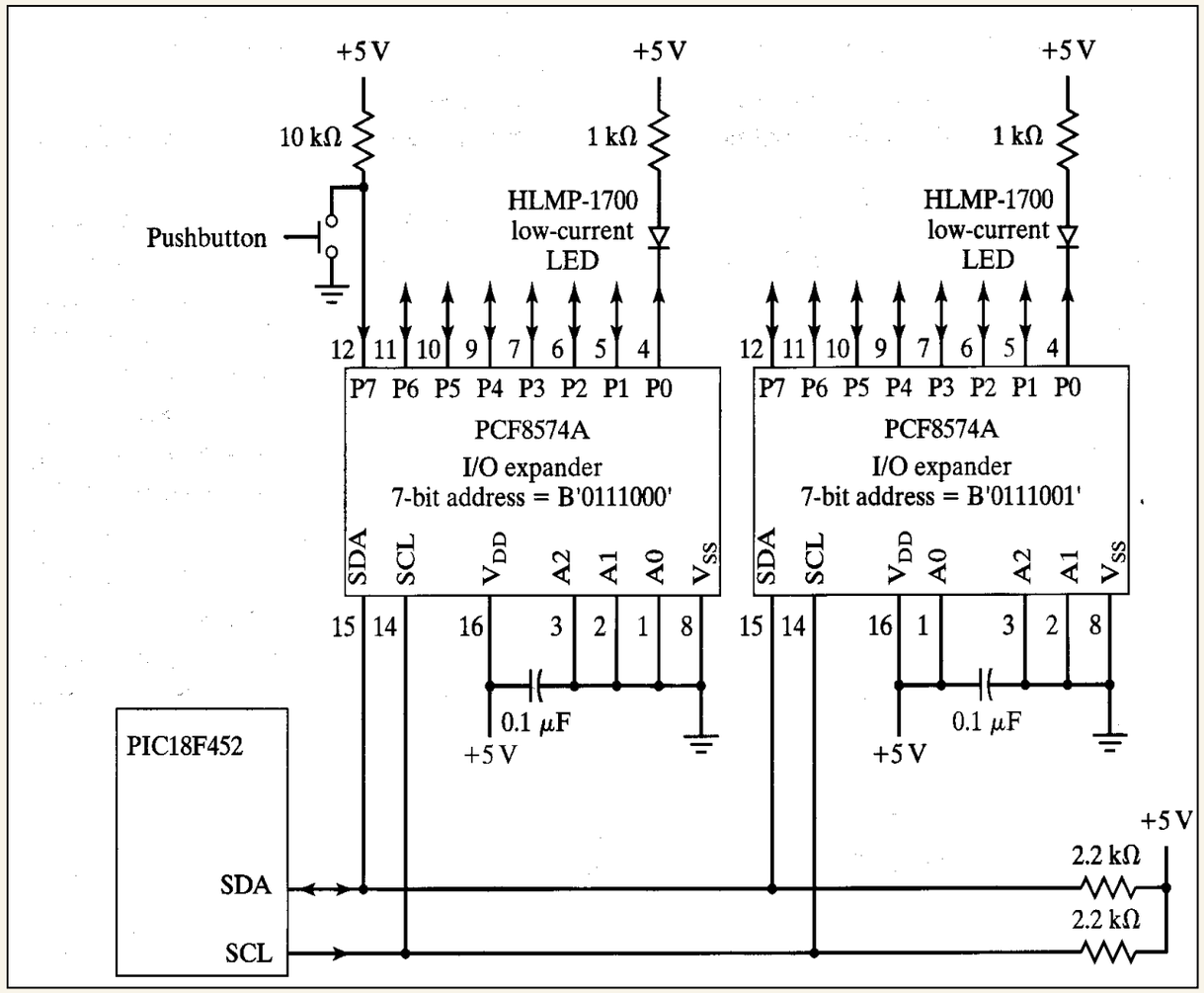

SMBus/I²C peripheral chips
PIC18F452
SDA SCL
SDA SCL
SDA SCL
SDA
SCL
2.2 kΩ
+5 V
2.2 kΩ

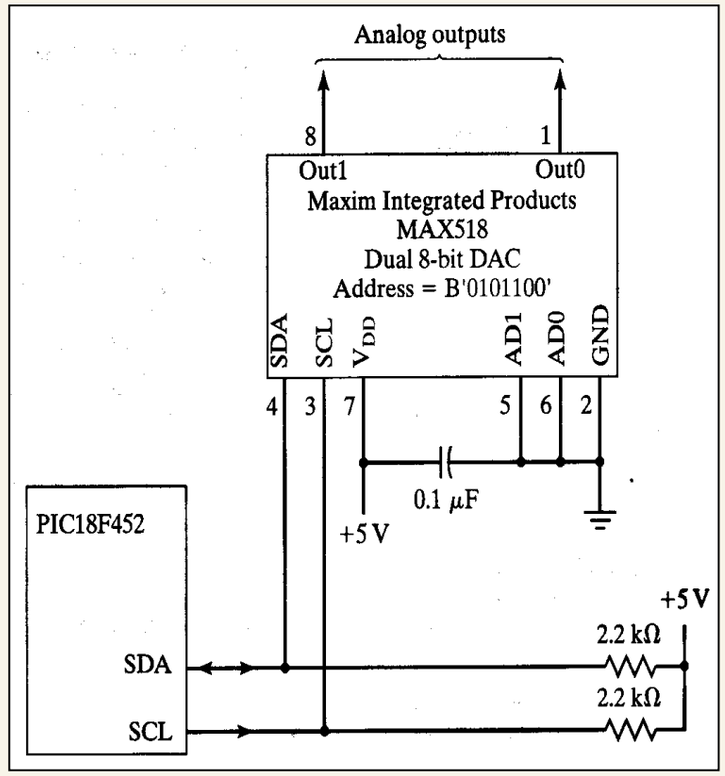

SDA
SCL
(a) START condition
SDA
SCL
(b) STOP condition

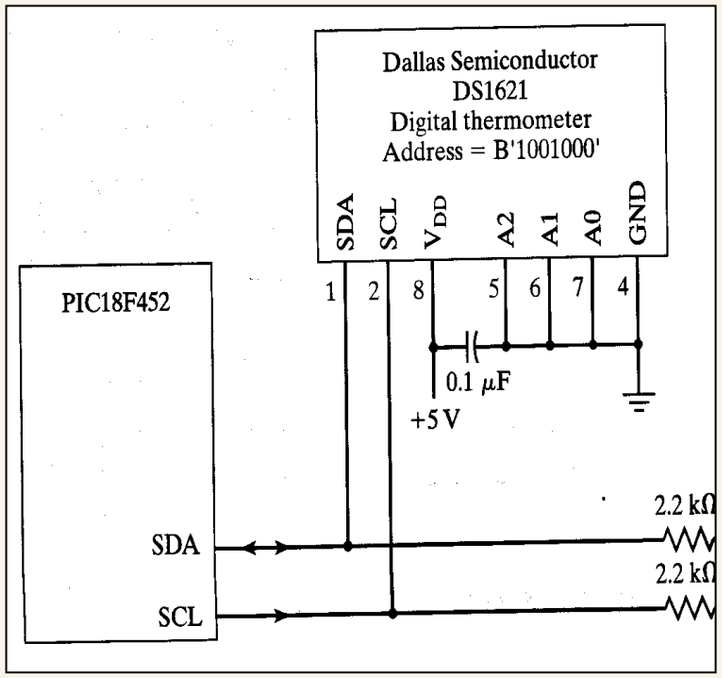

MSb
SDA
Slave address
R/W
1: Subsequent bytes will be sent by selected peripheral to PIC
0: Subsequent bytes will be sent by PIC to selected peripheral
Acceptable 7-bit slave addresses range from B'0001000' to B'1110111'


START condition
R/W = 0 for write
7-bit address of peripheral chip
Internal address, N, in peripheral
Data to be written into address N
Data to be written into address N + 1
Data to be written into address N + 2
STOP condition
SDA
S
W
ACK
P
Acknowledge by peripheral
(a) General format to write to several peripheral internal registers or addresses
START condition
R/W = 0 for write
RESTART condition
R/W = 1 for read
STOP condition
7-bit address of peripheral chip
Internal address, N, in peripheral
7-bit address of peripheral chip
Data read from address N
Data read from address N + 1
SDA
S
W
ACK
S
R
ACK
NO
P
Acknowledge by peripheral
Acknowledge by PIC
No acknowledge by PIC, sent out as a signal to terminate further transfers
(b) General format to read from several peripheral internal registers or addresses

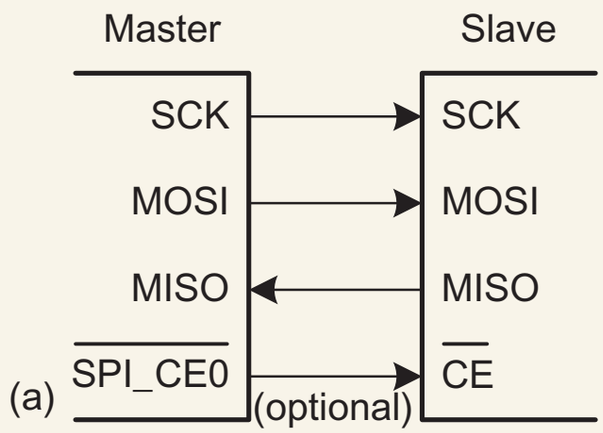
- I²C expansion circuits

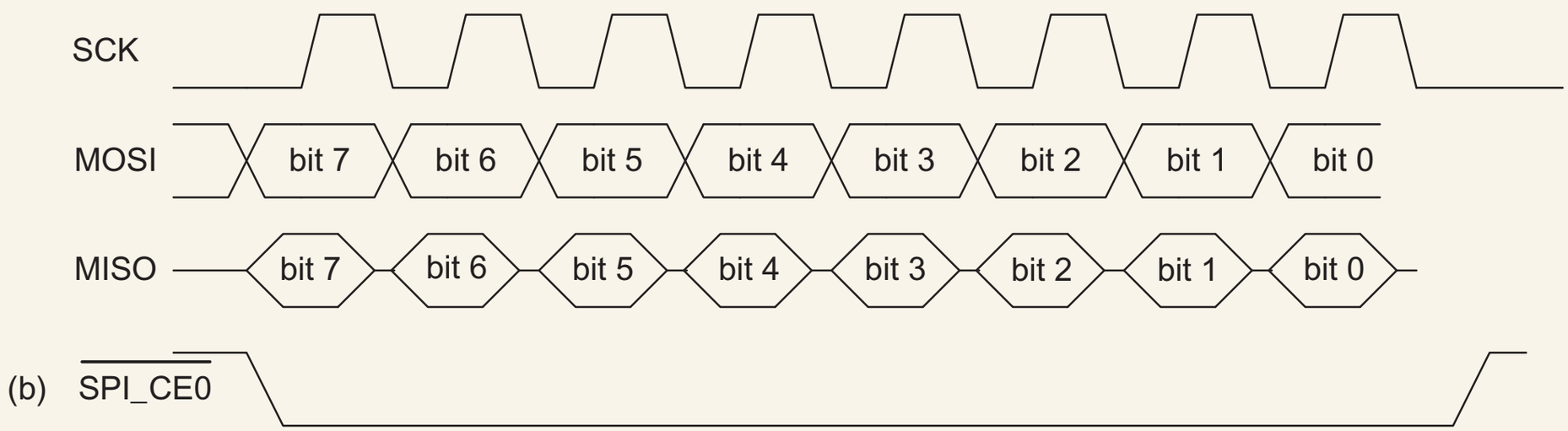






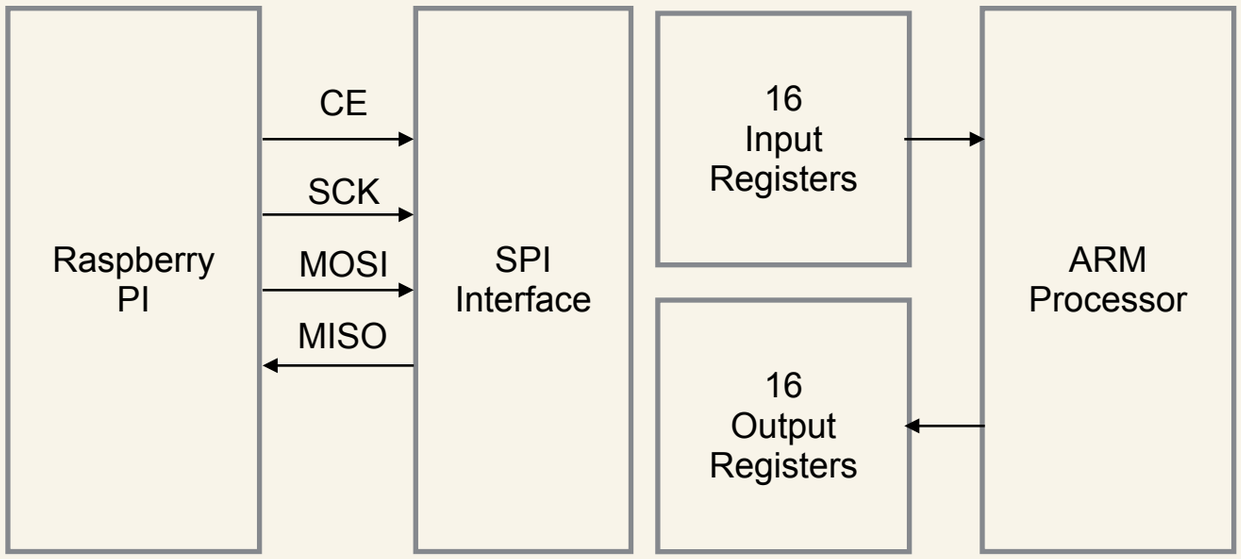
SPI


Master
Slave
SCK
SCK
MOSI
MOSI
MISO
MISO
SPI_CE0
CE
(a)
(optional)

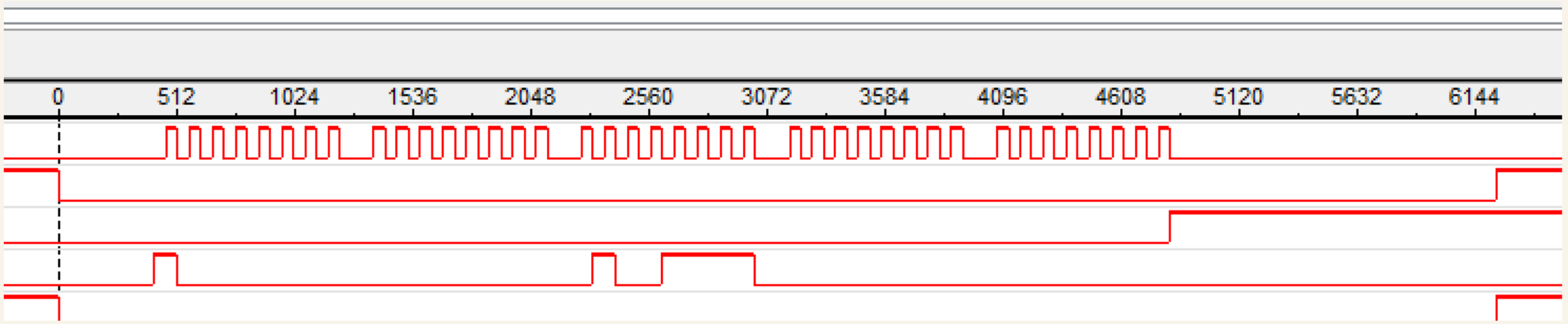

SCK
MOSI
bit 7
bit 6
bit 5
bit 4
bit 3
bit 2
bit 1
bit 0
MISO
bit 7
bit 6
bit 5
bit 4
bit 3
bit 2
bit 1
bit 0
(b)
SPI_CE0

SPI Interface

- 2 sets of 16 32-bit registers
- 5-byte packet
  - Byte 4
    - Bit 7 : R/W
    - Bit 6 - 4 : not used
    - Bit 3 - 0 : Address
  - Byte 3 - 0 : 32-bit Data

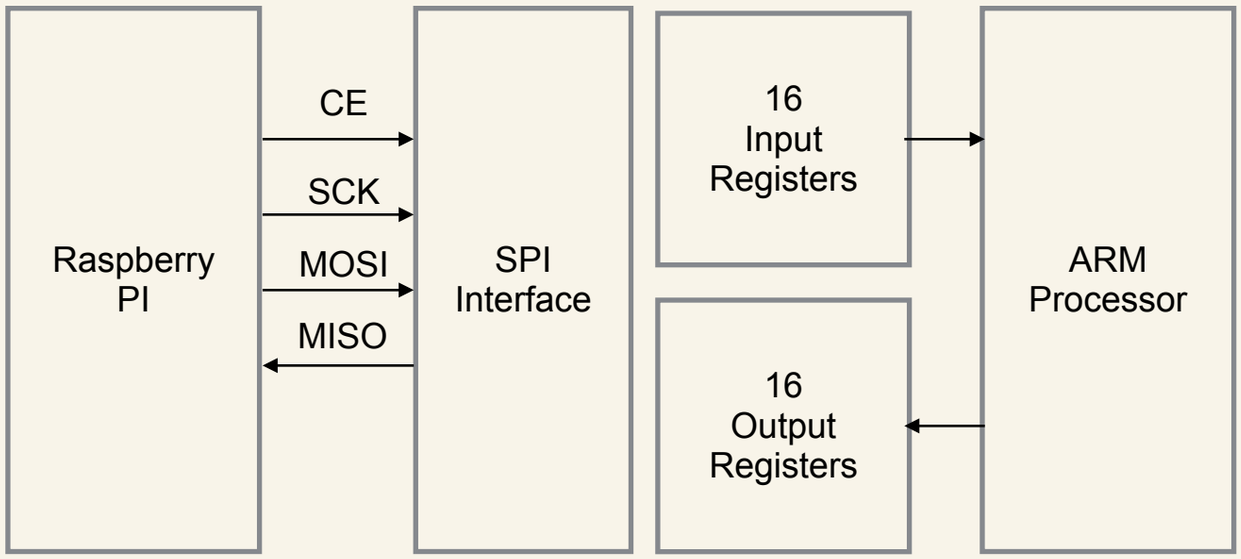



```
ToSPI = [0x80, 0x00, 0x4F, 0x00, 0x00]
FromSPI = MySPI_FPGA.xfer2(ToSPI)
```



- Input
  - 0 : Counting Frequency
  - 1 : 0x05
  - 2 : 0x04
- Output
  - 3 : 0x05 + 0x04 = 0x09

MyPI-Nano




```
MyARM_ResetPin = 19 # Pin 4 of connector = BCM19 = GPIO[1]

MySPI_FPGA = spidev.SpiDev()
MySPI_FPGA.open(0,0)
MySPI_FPGA.max_speed_hz = 500000

GPIO.setmode(GPIO.BCM)
GPIO.setwarnings(False)
GPIO.setup(MyARM_ResetPin, GPIO.OUT)

GPIO.output(MyARM_ResetPin, GPIO.HIGH)
sleep(0.1)
GPIO.output(MyARM_ResetPin, GPIO.LOW)
sleep(0.1)

ToSPI = [0x80, 0x00, 0x4F, 0x00, 0x00]
FromSPI = MySPI_FPGA.xfer2(ToSPI)

ToSPI = [0x81, 0x00, 0x00, 0x00, 0x05]
FromSPI = MySPI_FPGA.xfer2(ToSPI)

ToSPI = [0x82, 0x00, 0x00, 0x00, 0x04]
FromSPI = MySPI_FPGA.xfer2(ToSPI)

GPIO.output(MyARM_ResetPin, GPIO.HIGH)
sleep(0.1)
GPIO.output(MyARM_ResetPin, GPIO.LOW)
sleep(1)

ToSPI = [0x03, 0x00, 0x00, 0x00, 0x00]
FromSPI = MySPI_FPGA.xfer2(ToSPI)
print(FromSPI)
```

```
pi@raspberrypi:~/MyARM $ python3 MyARM_Pipelined.py
[0, 0, 0, 0, 9]
pi@raspberrypi:~/MyARM $
```

```
/*
    Loop on a counter and display the 8 LSB on the 8 LED of the DE0-Nano
    The counting frequency is fixed by the RaspberryPI via the SPI

    Address MAP : 0x0000 - 0x03FF : RAM (255 words of 32 bits)
                  0x0400 - 0x043F : SPI - 16 reg. of 32 bits
                  0x0500 :        : LED Reg

    SPI Registers : 0x0400 : SPI_Reg0 : Counting Frequency
                    0x0404 : SPI_Reg1
                    0x0408 : SPI_Reg2
                    0x040C : SPI_Reg3

    (the 16 MSB of the address are set to 0x0000 !)
*/
        STR R0, [R0, #0x0500]   @ Clear the LED
LOOP:   LDR R1, [R0, #0x0400]   @ R1 = counting frequency
DELAY:  SUBS R1, R1, #1         @ R1 = R1 - 1
        BNE DELAY               @ Branch if R1 != 0
        LDR R2, [R0, #0x0500]   @ R2 = led_reg
        ADD R2, R2, #1          @ R2 = R2 + 1
        STR R2, [R0, #0x0500]   @ led_reg = R2
/*
    Do SPI_Reg3 = SPI_Reg2 + SPI_Reg1
*/
        LDR R3, [R0, #0x0404]   @ R3 = SPI_Reg1
        LDR R4, [R0, #0x0408]   @ R4 = SPI_Reg2
        ADD R5, R3, R4          @ R5 = R2 + R4
        STR R5, [R0, #0x040C]   @ SPI_Reg3 = R5 = SPI_Reg2 + SPI_Reg1
        B   LOOP                @ infinite loop
```

MyDE0_Nano.sv

```
//=======================================================
//  MyARM
//=======================================================

   logic          clk, reset;
   logic [31:0]   WriteDataM, DataAdrM;
   logic          MemWriteM;
   logic [31:0]   PCF, InstrF, ReadDataM, ReadData_dmem, ReadData_spi;

   logic          cs_dmem, cs_led, cs_spi;
   logic [7:0]    led_reg;
   logic [31:0]   spi_data;

   assign clk   = CLOCK_50;
   assign reset = GPIO_0_PI[1];

   // Instantiate processor and memories
   arm arm(clk, reset, PCF, InstrF, MemWriteM, DataAdrM, WriteDataM, ReadDataM);
   imem imem(PCF, InstrF);
   dmem dmem(clk, cs_dmem, MemWriteM, DataAdrM, WriteDataM, ReadData_dmem);

   // Chip Select logic

   // Address MAP : 0x0000 - 0x03FF : RAM (255 words of 32 bits)
   //               0x0400 - 0x043F : SPI - 16 reg. of 32 bits
   //               0x0500 :        : LED Reg

   assign cs_dmem   = ~DataAdrM[11] & ~DataAdrM[10];                                  // 0x-0-- to 0x-3--
   assign cs_spi    = ~DataAdrM[11] &  DataAdrM[10] & ~DataAdrM[9] & ~DataAdrM[8];  // 0x-4--
   assign cs_led    = ~DataAdrM[11] &  DataAdrM[10] & ~DataAdrM[9] &  DataAdrM[8];  // 0x-5--

   // Read Data
   always_comb
      if (cs_dmem) ReadDataM = ReadData_dmem;
      else if (cs_spi) ReadDataM = spi_data;
      else if (cs_led) ReadDataM = {24'h000000, led_reg};
      else ReadDataM = 32'b0;

   // LED logic
   assign LED = led_reg;
   always_ff @(posedge clk)
      if (MemWriteM & cs_led) led_reg <= WriteDataM[7:0];

   // Testbench
   assign GPIO_1[33]    = MemWriteM;
   assign GPIO_1[15:0]  = WriteDataM[15:0];
   assign GPIO_1[31:16] = ReadDataM[15:0];
   assign GPIO_2[12:0]  = DataAdrM[12:0];
```

```
//=======================================================
//  SPI
//=======================================================

   logic          spi_clk, spi_cs, spi_mosi, spi_miso;

   spi_slave spi_slave_instance(
      .SPI_CLK    (spi_clk),
      .SPI_CS     (spi_cs),
      .SPI_MOSI   (spi_mosi),
      .SPI_MISO   (spi_miso),
      .Data_WE    (MemWriteM & cs_spi),
      .Data_Addr  (DataAdrM),
      .Data_Write (WriteDataM),
      .Data_Read  (spi_data),
      .Clk        (clk)
   );

   assign spi_clk       = GPIO_0_PI[11];  // SCLK = pin 16 = GPIO_11
   assign spi_cs        = GPIO_0_PI[9];   // CE0  = pin 14 = GPIO_9
   assign spi_mosi      = GPIO_0_PI[15];  // MOSI = pin 20 = GPIO_15

   assign GPIO_0_PI[13] = spi_cs ? 1'bz : spi_miso;  // MISO = pin 18 = GPIO_13
endmodule

//=======================================================
//  Memory
//=======================================================

module dmem(input logic clk, we, cs,
            input logic [31:0] a, wd,
            output logic [31:0] rd);

  logic [31:0] RAM[255:0];

assign rd = RAM[a[31:2]]; // word aligned

always_ff @(posedge clk)
    if (cs & we) RAM[a[31:2]] <= wd;
endmodule

module imem(input  logic [31:0] a,
            output logic [31:0] rd);

  logic [31:0] RAM[255:0];

initial $readmemh("MyProgram_Pipelined.hex",RAM);
assign rd = RAM[a[31:2]]; // word aligned

endmodule
```

MySPI.sv

```
//------------------------------------------------------------------
//
// SPI Module
//

module spi_slave(
   input  logic         SPI_CLK,
   input  logic         SPI_CS,
   input  logic         SPI_MOSI,
   output logic         SPI_MISO,
   input  logic         Data_WE,
   input  logic[31:0]   Data_Addr,
   input  logic[31:0]   Data_Write,
   output logic[31:0]   Data_Read,
   input  logic         Clk
);

   logic [39:0] SPI_reg;


//---Input Registers -(from PI to ARM) ------------------------

   logic [31:0] mosiRAM[15:0];
   logic        mosiRAM_we;

   assign Data_Read = mosiRAM[Data_Addr[5:2]];

   always_ff @(posedge Clk) begin
      if (mosiRAM_we) mosiRAM[SPI_reg[35:32]] <= SPI_reg[31:0];
   end

//--- Output Registers - (from ARM to PI)----------------------

   logic [31:0] misoRAM[15:0];
   logic [31:0] misoRAM_read;

   assign misoRAM_read = misoRAM[SPI_reg[3:0]];

   always_ff @(posedge Clk) begin
      if (Data_WE) misoRAM[Data_Addr[5:2]] <= Data_Write;
   end

//---SPI Sysnchronization -------------------------------------

   logic SPI_CLK_sync;
   logic SPI_CS_sync;

   always_ff @(posedge Clk) begin
      SPI_CLK_sync <= SPI_CLK;
      SPI_CS_sync  <= SPI_CS;
   end
```

```
//--- SPI FSM -----------------------------------------------

   typedef enum logic [1:0] {S0,S1,S2,S3} statetype;
   statetype state, nextstate;

   logic [5:0] SPI_cnt;
   logic       SPI_cnt_reset, SPI_cnt_inc;
   logic       SPI_reg_reset, SPI_reg_shift, SPI_reg_load;
   logic       MISO_we, MISO_reset;

// State Register & Bit counter & SPI Register & MISO

   always_ff @(posedge Clk) begin

      if (SPI_CS_sync)        state <= S0;
      else                    state <= nextstate;

      if (SPI_cnt_reset)      SPI_cnt <= 6'b0;
      else if (SPI_cnt_inc)   SPI_cnt <= SPI_cnt + 6'b1;

      if (SPI_reg_reset)      SPI_reg <= 40'b0;
      else if (SPI_reg_shift) SPI_reg <= {SPI_reg[38:0], SPI_MOSI};
      else if (SPI_reg_load)  SPI_reg <= {misoRAM_read, SPI_reg[7:0]};

      if (MISO_reset)         SPI_MISO <= 0;
      else if (SPI_reg_load)  SPI_MISO <= misoRAM_read[31];
      else if (MISO_we)       SPI_MISO <= SPI_reg[39];

   end

// Next State Logic

   always_comb begin

      // Default value
      nextstate = state;
      SPI_cnt_reset = 0; SPI_cnt_inc = 0;
      SPI_reg_reset = 0; SPI_reg_shift = 0; SPI_reg_load = 0;
      MISO_we = 0; MISO_reset = 0;
      mosiRAM_we = 0;

      case (state)
         S0 : if (~SPI_CS_sync) begin        // negedge of SPI_CS
                 SPI_cnt_reset = 1;
                 SPI_reg_reset = 1;
                 MISO_reset    = 1;
                 nextstate = S1;
              end

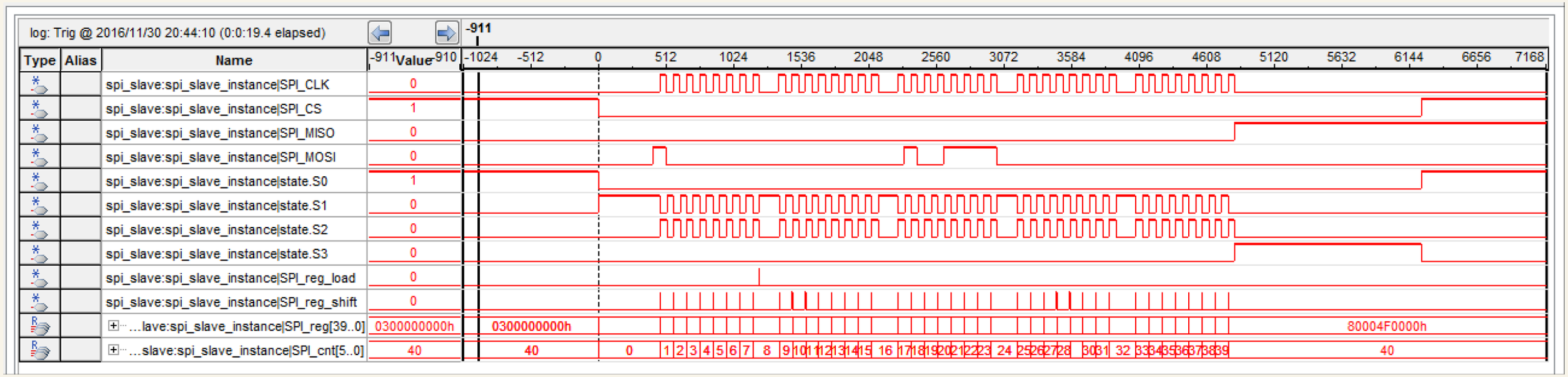
         S1 : if (SPI_CLK_sync) begin        // posedge of SPI_CLK
                 SPI_reg_shift = 1;
                 SPI_cnt_inc   = 1;
                 nextstate = S2;
              end

         S2 : if (~SPI_CLK_sync) begin       // negedge of SPI_CLK
                 MISO_we = 1;
                 if (SPI_cnt == 8) SPI_reg_load = 1;
                 if (SPI_cnt == 40) begin
                    if (SPI_reg[39] == 1) mosiRAM_we = 1;
                    nextstate = S3;
                 end
                 else nextstate = S1;
              end

         S3 : if (SPI_CS_sync) begin         // posedge of SPI_CS
                 nextstate = S0;
              end
      endcase
   end

endmodule

//------------------------------------------------------------------
```




```
ToSPI = [0x80, 0x00, 0x4F, 0x00, 0x00]
FromSPI = MySPI_FPGA.xfer2(ToSPI)
```

```
// Next State Logic

   always_comb begin

      // Default value
      nextstate = state;
      SPI_cnt_reset = 0; SPI_cnt_inc = 0;
      SPI_reg_reset = 0; SPI_reg_shift = 0; SPI_reg_load = 0;
      MISO_we = 0; MISO_reset = 0;
      mosiRAM_we = 0;

      case (state)
         S0 : if (~SPI_CS_sync) begin        // negedge of SPI_CS
                  SPI_cnt_reset = 1;
                  SPI_reg_reset = 1;
                  MISO_reset    = 1;
                  nextstate = S1;
              end

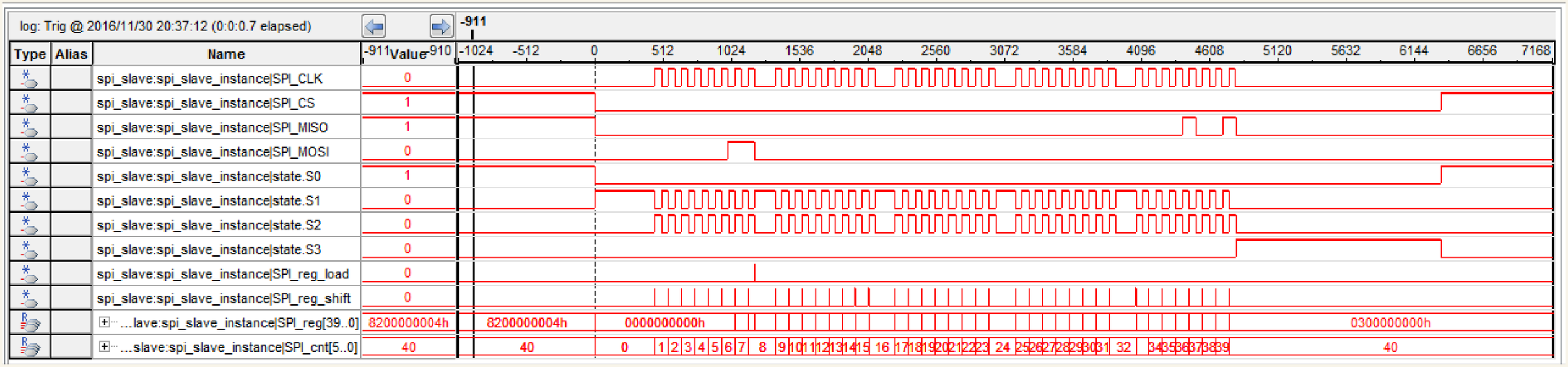
         S1 : if (SPI_CLK_sync) begin        // posedge of SPI_CLK
                  SPI_reg_shift = 1;
                  SPI_cnt_inc   = 1;
                  nextstate = S2;
              end

         S2 : if (~SPI_CLK_sync) begin       // negedge of SPI_CLK
                  MISO_we = 1;
                  if (SPI_cnt == 8) SPI_reg_load = 1;
                  if (SPI_cnt == 40) begin
                     if (SPI_reg[39] == 1) mosiRAM_we = 1;
                     nextstate = S3;
                  end
                  else nextstate = S1;
              end

         S3 : if (SPI_CS_sync) begin         // posedge of SPI_CS
                  nextstate = S0;
              end
      endcase
   end

endmodule

//----------------------------------------------------------------
```




```
ToSPI = [0x03, 0x00, 0x00, 0x00, 0x00]
FromSPI = MySPI_FPGA.xfer2(ToSPI)
print(FromSPI)
```

```
pi@raspberrypi:~/MyARM $ python3 MyARM_Pipelined.py
[0, 0, 0, 0, 9]
pi@raspberrypi:~/MyARM $
```

```
// Next State Logic

   always_comb begin

      // Default value
      nextstate = state;
      SPI_cnt_reset = 0; SPI_cnt_inc = 0;
      SPI_reg_reset = 0; SPI_reg_shift = 0; SPI_reg_load = 0;
      MISO_we = 0; MISO_reset = 0;
      mosiRAM_we = 0;

      case (state)
         S0 : if (~SPI_CS_sync) begin        // negedge of SPI_CS
                  SPI_cnt_reset = 1;
                  SPI_reg_reset = 1;
                  MISO_reset    = 1;
                  nextstate = S1;
              end

         S1 : if (SPI_CLK_sync) begin        // posedge of SPI_CLK
                  SPI_reg_shift = 1;
                  SPI_cnt_inc   = 1;
                  nextstate = S2;
              end

         S2 : if (~SPI_CLK_sync) begin       // negedge of SPI_CLK
                  MISO_we = 1;
                  if (SPI_cnt == 8) SPI_reg_load = 1;
                  if (SPI_cnt == 40) begin
                     if (SPI_reg[39] == 1) mosiRAM_we = 1;
                     nextstate = S3;
                  end
                  else nextstate = S1;
              end

         S3 : if (SPI_CS_sync) begin         // posedge of SPI_CS
                  nextstate = S0;
              end
      endcase
   end

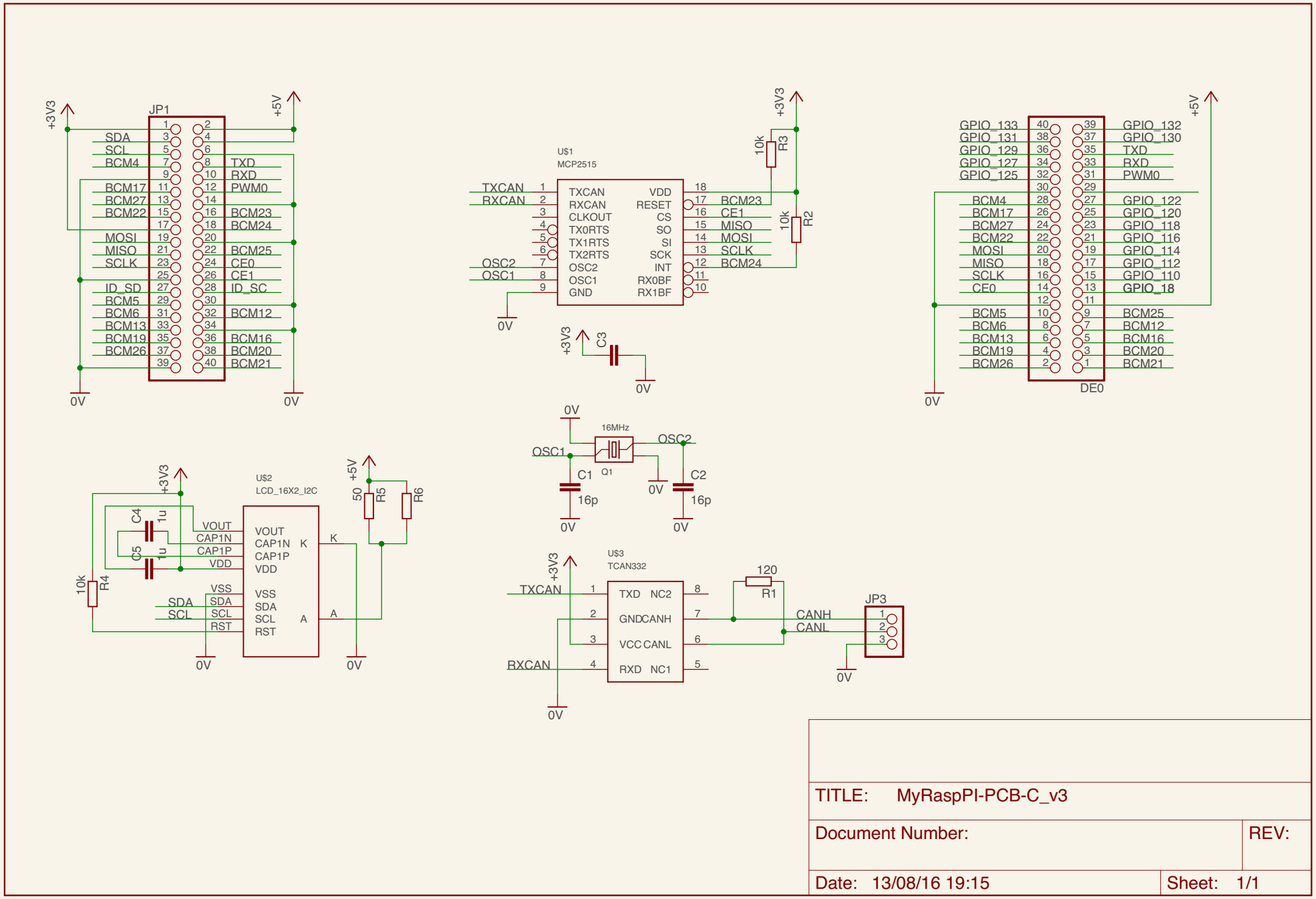
endmodule

//---------------------------------------------------------------
```

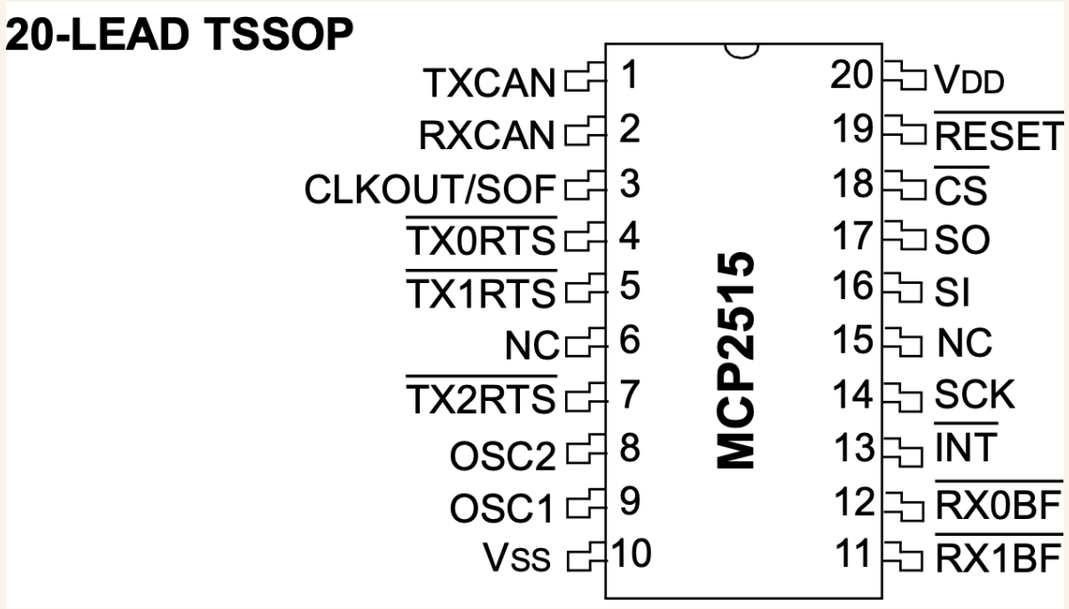
CAN

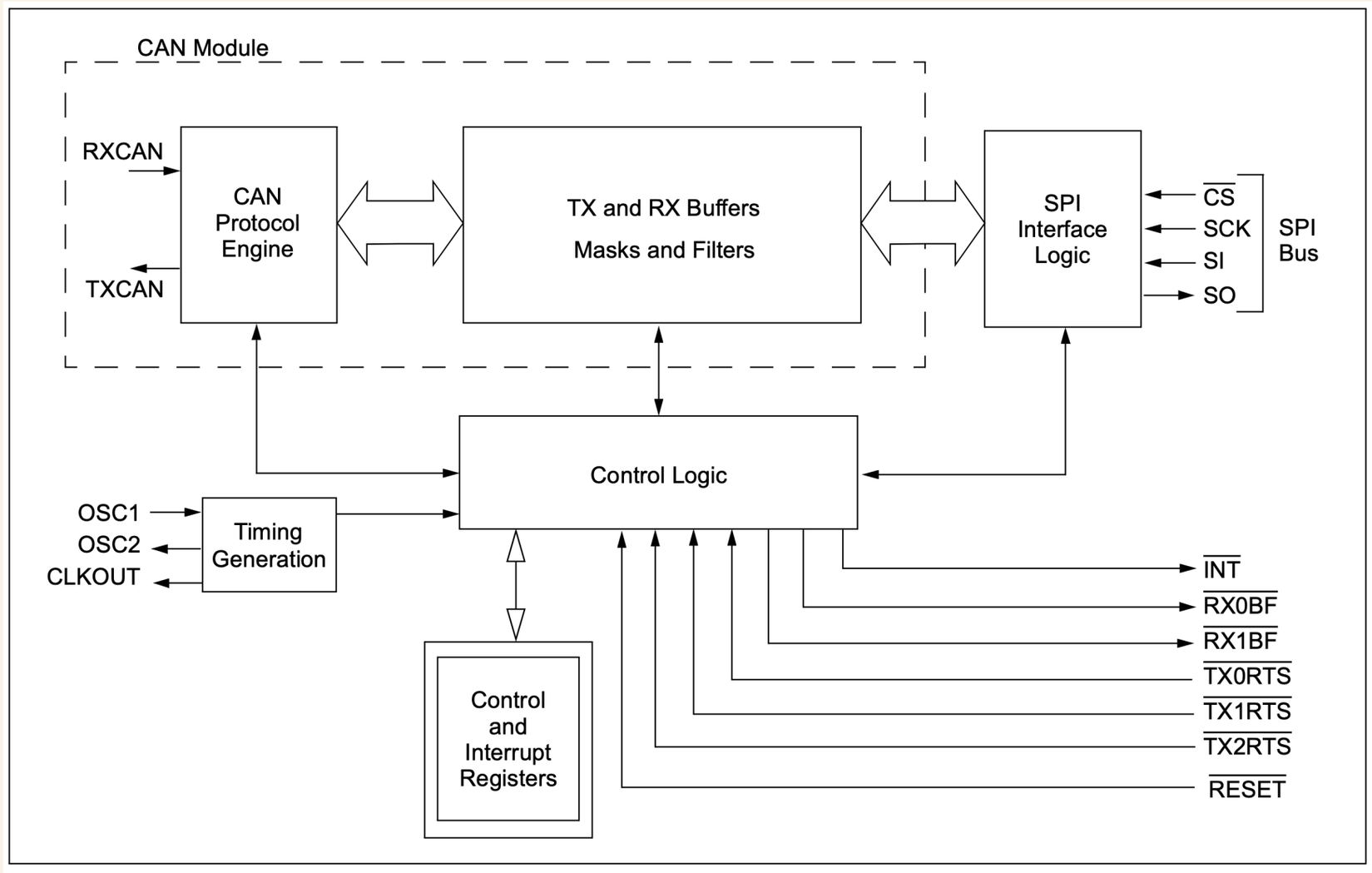
- Field Domain
  - Production Automation
  - Mobile Systems
  - Building Automation
  - Embedded Systems
- Main Characteristics of CAN
  - Linear topology
  - Number of nodes not limited
  - 1 Mbits/s for 40m network, 80 Kbits/s for 1000m network
  - Message-oriented protocol
  - Prioritization of messages
  - Multimaster capability
  - Loss-free bus arbitration
  - Short frame length
  - High data integrity, very short error recovery time
  - Network-wide data consistence
  - Detection and de-activation of defective nodes
  - Short latency time for high-priority messages
  - International standardization
  - Higher Layer protocol implementation (CANopen DeviceNet)
  - CAN assocation (www.can-cia.de)

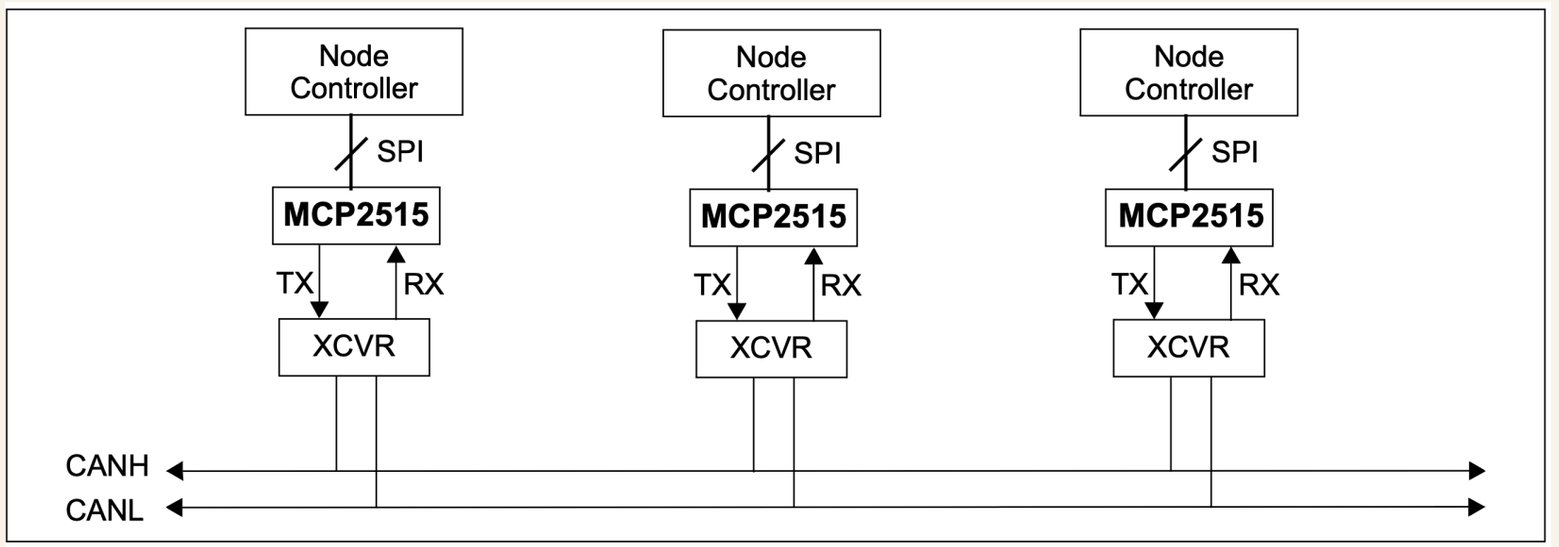
- Interface board between Raspberry PI and DE0-Nano

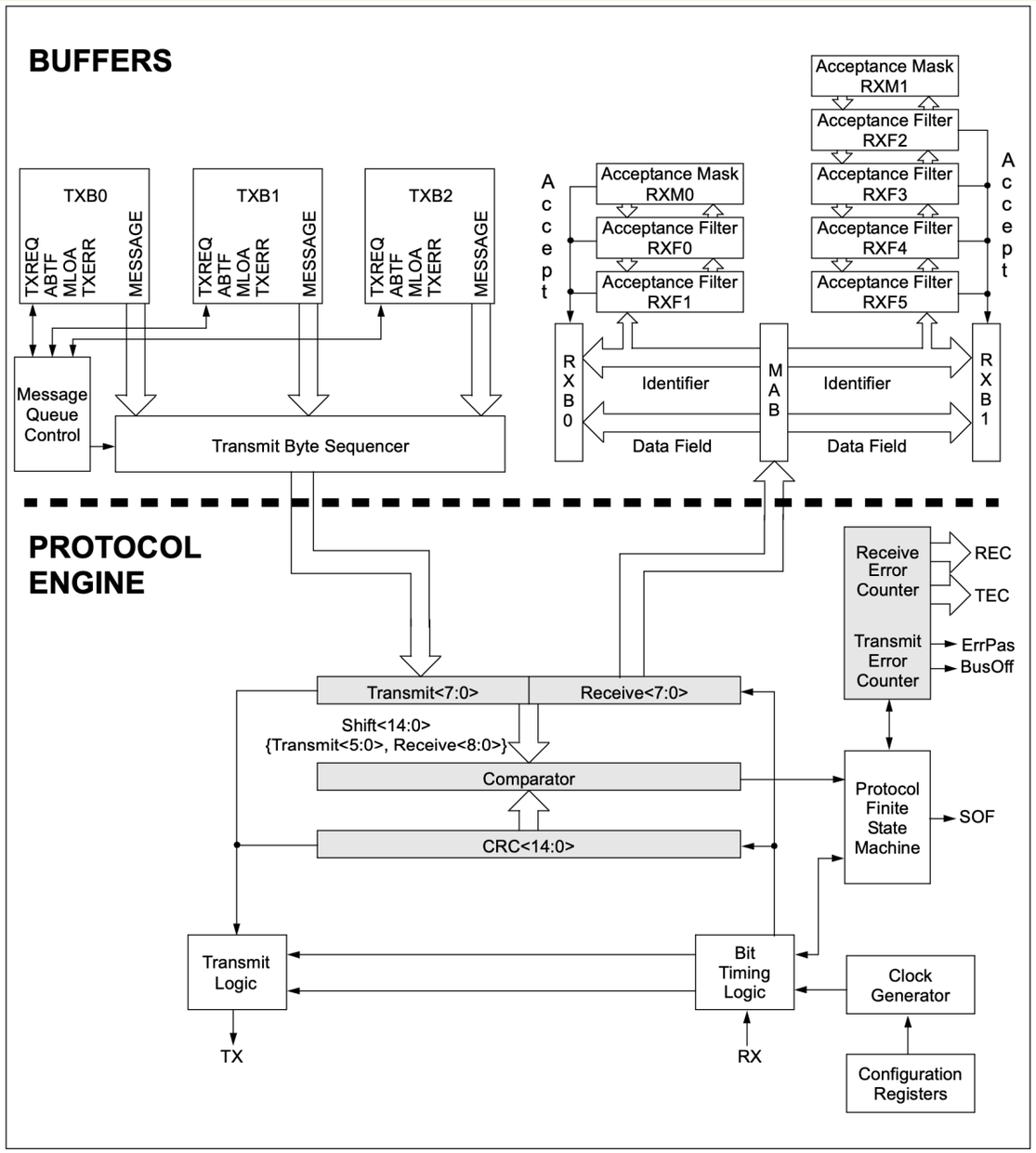


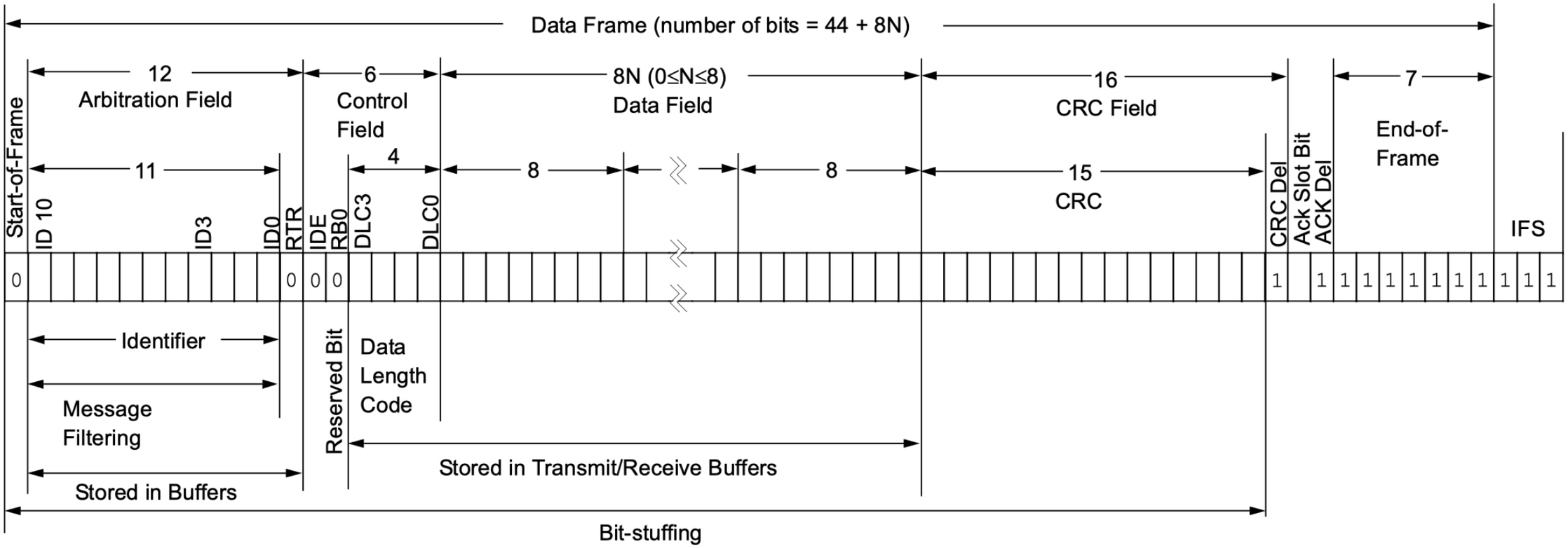
- MCP2515 - CAN Controller with SPI Interface






Node Controller
SPI
MCP2515
TX
RX
XCVR
Node Controller
SPI
MCP2515
TX
RX
XCVR
Node Controller
SPI
MCP2515
TX
RX
XCVR
CANH
CANL


BUFFERS
TXB0
TXREQ
ABTF
MLOA
TXERR
MESSAGE
TXB1
TXREQ
ABTF
MLOA
TXERR
MESSAGE
TXB2
TXREQ
ABTF
MLOA
TXERR
MESSAGE
Message Queue Control
Transmit Byte Sequencer
Acceptance Mask RXM0
Acceptance Filter RXF0
Acceptance Filter RXF1
Acceptance Mask RXM1
Acceptance Filter RXF2
Acceptance Filter RXF3
Acceptance Filter RXF4
Acceptance Filter RXF5
Accept
RXB0
Identifier
Data Field
MAB
Identifier
Data Field
RXB1
PROTOCOL ENGINE
Receive Error Counter
REC
TEC
Transmit Error Counter
ErrPas
BusOff
Transmit<7:0>
Receive<7:0>
Shift<14:0>
{Transmit<5:0>, Receive<8:0>}
Comparator
CRC<14:0>
Protocol Finite State Machine
SOF
Transmit Logic
Bit Timing Logic
Clock Generator
TX
RX
Configuration Registers


Data Frame (number of bits = 44 + 8N)
12
Arbitration Field
6
Control Field
8N (0≤N≤8)
Data Field
16
CRC Field
7
End-of-Frame
Start-of-Frame
ID 10
ID3
ID0
RTR
IDE
RB0
DLC3
DLC0
11
4
8
8
15
CRC
CRC Del
Ack Slot Bit
ACK Del
IFS
0
0
0
0
1
1
1
1
1
1
1
1
1
1
Identifier
Message Filtering
Stored in Buffers
Reserved Bit
Data Length Code
Stored in Transmit/Receive Buffers
Bit-stuffing

- Electrical Transmission Media
  - Differentially driven pair of wires

